RISC-V 指令異常偵錯實例

原創 qinyunti 嵌入式 Lee 2023 年 05 月 07 日 07:01 湖南

前言

本文以一個簡單的實例介紹 RISC-V 指令異常的偵錯過程，思路都是一樣的，遇到其他情況時分析過程也類似。
相關內容參考《riscv-privileged-20211203.pdf》

過程

現像是程序執行後進入了異常中斷,可以通過 GDB 的 bt 命令看到

```
#12      0x02002e9c      in      exception      ()      at
src/lib/riscv/src/exception.c:55
#13 0x02002b40 in is_exception ()
Backtrace stopped: frame did not save the PC
(gdb)
```

既然是進入了異常中斷,那麼就需要確認到底是什麼異常，
這可以通過 **mcause** 暫存器查看

```
(gdb) info reg mcause
mcause 0x2      0x2
(gdb)
```

可以看到是非法指令異常
那麼我們就搜尋文件的 Illegal instruction 可以查看到所有可能導致 Illegal instruction 的原因。

Interrupt	Exception Code	Description
1	0	*Reserved*
1	1	Supervisor software interrupt
1	2	*Reserved*
1	3	Machine software interrupt
1	4	*Reserved*
1	5	Supervisor timer interrupt
1	6	*Reserved*
1	7	Machine timer interrupt
1	8	*Reserved*
1	9	Supervisor external interrupt
1	10	*Reserved*
1	11	Machine external interrupt
1	12–15	*Reserved*
1	≥16	*Designated for platform use*
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10	*Reserved*
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	14	*Reserved*
0	15	Store/AMO page fault
0	16–23	*Reserved*
0	24–31	*Designated for custom use*
0	32–47	*Reserved*
0	48–63	*Designated for custom use*
0	≥64	*Reserved*

Table 3.6: Machine cause register (`mcause`) values after trap.

我們搜到以下資訊，即 **mtval** 暫存器保存了異常指令，**mepc** 指向了異常指令

3.1.16 Machine Trap Value Register (mtval)

The mtval register is an MXLEN-bit read-write register formatted as shown in Figure 3.23. When a trap is taken into M-mode, mtval is either set to zero or written with exception-specific information to assist software in handling the trap. Otherwise, mtval is never written by the implementation, though it may be explicitly written by software. The hardware platform will specify which exceptions must set mtval informatively and which may unconditionally set it to zero. If the hardware platform specifies that no exceptions set mtval to a nonzero value, then mtval is read-only zero.

If mtval is written with a nonzero value when a breakpoint, address-misaligned, access-fault, or page-fault exception occurs on an instruction fetch, load, or store, then mtval will contain the faulting virtual address.

When page-based virtual memory is enabled, mtval *is written with the faulting virtual address, even for physical-memory access-fault exceptions. This design reduces datapath cost for most implementations, particularly those with hardware page-table walkers.*

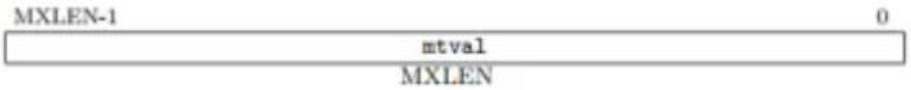

Figure 3.23: Machine Trap Value register.

If mtval is written with a nonzero value when a misaligned load or store causes an access-fault or page-fault exception, then mtval will contain the virtual address of the portion of the access that caused the fault.

If mtval is written with a nonzero value when an instruction access-fault or page-fault exception occurs on a system with variable-length instructions, then mtval will contain the virtual address of the portion of the instruction that caused the fault, while mepc will point to the beginning of the instruction.

The mtval register can optionally also be used to return the faulting instruction bits on an illegal instruction exception (mepc points to the faulting instruction in memory). If mtval is written with a nonzero value when an illegal-instruction exception occurs, then mtval will contain the shortest of:

- the actual faulting instruction
- the first ILEN bits of the faulting instruction
- the first MXLEN bits of the faulting instruction

The value loaded into mtval on an illegal-instruction exception is right-justified and all unused upper bits are cleared to zero.

Capturing the faulting instruction in mtval *reduces the overhead of instruction emulation, potentially avoiding several partial instruction loads if the instruction is misaligned, and likely data cache misses or slow uncached accesses when loads are used to fetch the instruction into a data register. There is also a problem of atomicity if another agent is manipulating the instruction memory, as might occur in a dynamic translation system.*

A requirement is that the entire instruction (or at least the first MXLEN bits) are fetched into `mtval` *before taking the trap. This should not constrain implementations, which would typically fetch the entire instruction before attempting to decode the instruction, and avoids complicating software handlers.*

A value of zero in `mtval` *signifies either that the feature is not supported, or an illegal zero instruction was fetched. A load from the instruction memory pointed to by* `mepc` *can be used to distinguish these two cases (or alternatively, the system configuration information can be interrogated to install the appropriate trap handling before runtime).*

For other traps, `mtval` is set to zero, but a future standard may redefine `mtval`'s setting for other traps.

If `mtval` is not read-only zero, it is a **WARL** register that must be able to hold all valid virtual addresses and the value zero. It need not be capable of holding all possible invalid addresses. Prior to writing `mtval`, implementations may convert an invalid address into some other invalid address that `mtval` is capable of holding. If the feature to return the faulting instruction bits is implemented, `mtval` must also be able to hold all values less than 2^N, where N is the smaller of MXLEN and ILEN.

嵌入式Lee

可以看到 `mepc` 的內容是 `0`，那麼猜測應該是函數指針未初始化直接呼叫導致的

```
(gdb) info reg mtval
mtval 0x0      0x0
(gdb) info reg mepc
mepc 0x0      0x0
(gdb)
```

到這裡基本就確認了方向了，可以重點看哪些地方有函數指針，或者逐步註釋函數，或者逐步斷點定位即可。這裡很快就確認了是如下程式碼導致

```
int xxx_ioctl(unsigned int dev_id, unsigned int cmd, void *data)
{
  if (dev_id >= xxx_drv.dev_num)
    return -1;
  return xxx_drv.ops.ioctl(&(xxx_drv.dev[dev_id]), cmd, data);
}
```

查看函數指針正好是 `0`

```
(gdb) p xxx_drv.ops.ioctl
$1 = (int (*)(struct xxx_dev_s *, unsigned int, void *)) 0x0
(gdb)
```

回溯程式碼確認了是某個外設沒有初始化成功則這個回呼函數沒有初始化。原因就定位了。

總結

對於異常的偵錯可以參考手冊《riscv-privileged-20211203.pdf》，從異常原因入手，逐漸反推，確認異常觸發點然後確定原因。